

SGMS Auth Microservice API Contract

API Reference Documentation

Version 1.0.0

January 3, 2026

1. Introduction

The SGMS Auth Microservice provides authentication and authorization services following RESTful principles. This API supports user registration, email verification, login/logout functionality, token management, and password recovery mechanisms.

1.1 Base URL

All endpoints are relative to the base URL of the authentication service.

1.2 Authentication

The API uses cookies for authentication. Most endpoints require an Authorization header with a Cookie token:

```
Cookie: <access_token> || <refresh_token>
```

2. API Endpoints

2.1 Welcome Endpoint

- **Endpoint:** GET /
- **Description:** Welcome message endpoint.
- **Response:** 200 OK with welcome message

2.2 Authentication Endpoints

2.2.1 User Registration

- **Endpoint:** POST /api/v1/auth/register
- **Description:** Register a new user
- **Request Body:**

```
{  
  "email": "string",  
  "password": "string"  
}
```

- **Responses:**

- **201 Created:** Registration successful
- **422 Validation Error:** Invalid input data

2.2.2 Email Verification

- **Endpoint:** POST /api/v1/auth/verify-email
- **Description:** Verify email address using token
- **Request Body:**

```
{  
  "token": "string"  
}
```

- **Responses:**
 - **200 OK:** Email verified successfully
 - **422 Validation Error:** Invalid token

2.2.3 Resend Verification Email

- **Endpoint:** POST /api/v1/auth/resend-verification
- **Description:** Resend email verification
- **Request Body:**

```
{  
  "email": "string"  
}
```

- **Responses:**
 - **200 OK:** Verification email resent
 - **422 Validation Error:** Invalid email

2.2.4 User Login

- **Endpoint:** POST /api/v1/auth/login
- **Description:** Authenticate user and return tokens
- **Request Body:**

```
{  
  "email": "string",  
  "password": "string"  
}
```

- **Responses:**
 - **200 OK:** Returns access and refresh tokens
 - **422 Validation Error:** Invalid credentials

2.2.5 Refresh Access Token

- **Endpoint:** POST /api/v1/auth/refresh-token
- **Description:** Generate new access token using refresh token
- **Responses:**
 - **200 OK:** Returns new access token

2.2.6 User Logout

- **Endpoint:** POST /api/v1/auth/logout
- **Description:** Invalidate user session
- **Responses:**
 - **200 OK:** Logout successful

2.2.7 Forgot Password

- **Endpoint:** POST /api/v1/auth/forgot-password
- **Description:** Request password reset email
- **Request Body:**

```
{
  "email": "string"
}
```
- **Responses:**
 - **200 OK:** Reset email sent (if email exists)
 - **422 Validation Error:** Invalid email

2.2.8 Reset Password

- **Endpoint:** POST /api/v1/auth/reset-password
- **Description:** Reset password using token
- **Request Body:**

```
{
  "token": "string",
  "new_password": "string"
}
```
- **Responses:**
 - **200 OK:** Password reset successful
 - **422 Validation Error:** Invalid token or password

2.3 Token Verification

- **Endpoint:** POST /api/v1/auth/verify-token
- **Description:** Verify access token and return user context
- **Responses:**
 - **200 OK:** Token is valid

3. Data Models

3.1 UserRequest

```
{
  "email": "string (email format)",
  "password": "string"
}
```

3.2 VerifyTokenRequest

```
{
  "token": "string"
}
```

3.3 ResendVerificationRequest

```
{
  "email": "string (email format)"
}
```

3.4 ForgotPasswordRequest

```
{
  "email": "string (email format)"
}
```

3.5 ResetPasswordRequest

```
{
  "token": "string",
  "new_password": "string"
}
```

4. Error Handling

4.1 HTTPValidationError

```
{
  "detail": [
    {
      "loc": ["string" or "integer"],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

4.2 Common HTTP Status Codes

- **200 OK:** Request successful
- **201 Created:** Resource created successfully
- **400 Bad Request:** Invalid request parameters
- **401 Unauthorized:** Authentication required
- **403 Forbidden:** Insufficient permissions
- **422 Unprocessable Entity:** Validation error
- **500 Internal Server Error:** Server error

5. Security Considerations

- All sensitive endpoints require authentication
- Passwords are hashed using bcrypt or similar algorithm
- JWT tokens have expiration times
- Refresh tokens are securely stored and rotated
- Email verification prevents unauthorized account creation
- Password reset tokens are single-use and time-limited

6. Rate Limiting

API requests are subject to rate limiting to prevent abuse. The current limits are:

- Authentication endpoints: 5 requests per minute per IP
- Other endpoints: 60 requests per minute per user

7. Versioning

The API uses URL versioning (`/api/v1/`). Future versions will maintain backward compatibility where possible.

8. Implementation Notes

- All email fields must be valid email addresses
- Password requirements: minimum 8 characters, at least one uppercase, one lowercase, one number, and one special character
- Tokens are Base64 encoded and include expiration timestamps
- Refresh tokens are stored securely and invalidated after use
- Email verification tokens expire after 24 hours
- Password reset tokens expire after 1 hour